

INVENTORA

ITE 108 - ADVANCED DATABASE MANAGEMENT SYSTEM





I. PROJECT BACKGROUND

Inventora is an advanced inventory management system designed to track and manage products, stock adjustments, and user activities. It ensures efficient stock monitoring, audit tracking, and role-based access to maintain system integrity and accountability. The system is designed to provide detailed insights into stock levels, inventory adjustments, and user activity logs through robust database functionalities such as views, materialized views, functions, and indexing.



A. TRANSACTIONS IN THE SYSTEM

- **Product Management:**

Users can add, update, and view product details, including product name, category, stock quantity, and price per unit.

- **Inventory Adjustments:**

Users can adjust stock quantities (adding or removing stock) via transactions recorded in the inventory_adjustments table. Adjustments include details of the adjusted quantity, type of adjustment, and the user responsible.

- **Report Generation:**

The system provides reports through views and materialized views, including product summaries, critical stock levels, and activity logs.

- **Activity Logging:**

All user actions are logged in the activity_logs table, tracking changes made to the database, including the tables and columns affected, user role, and timestamp.



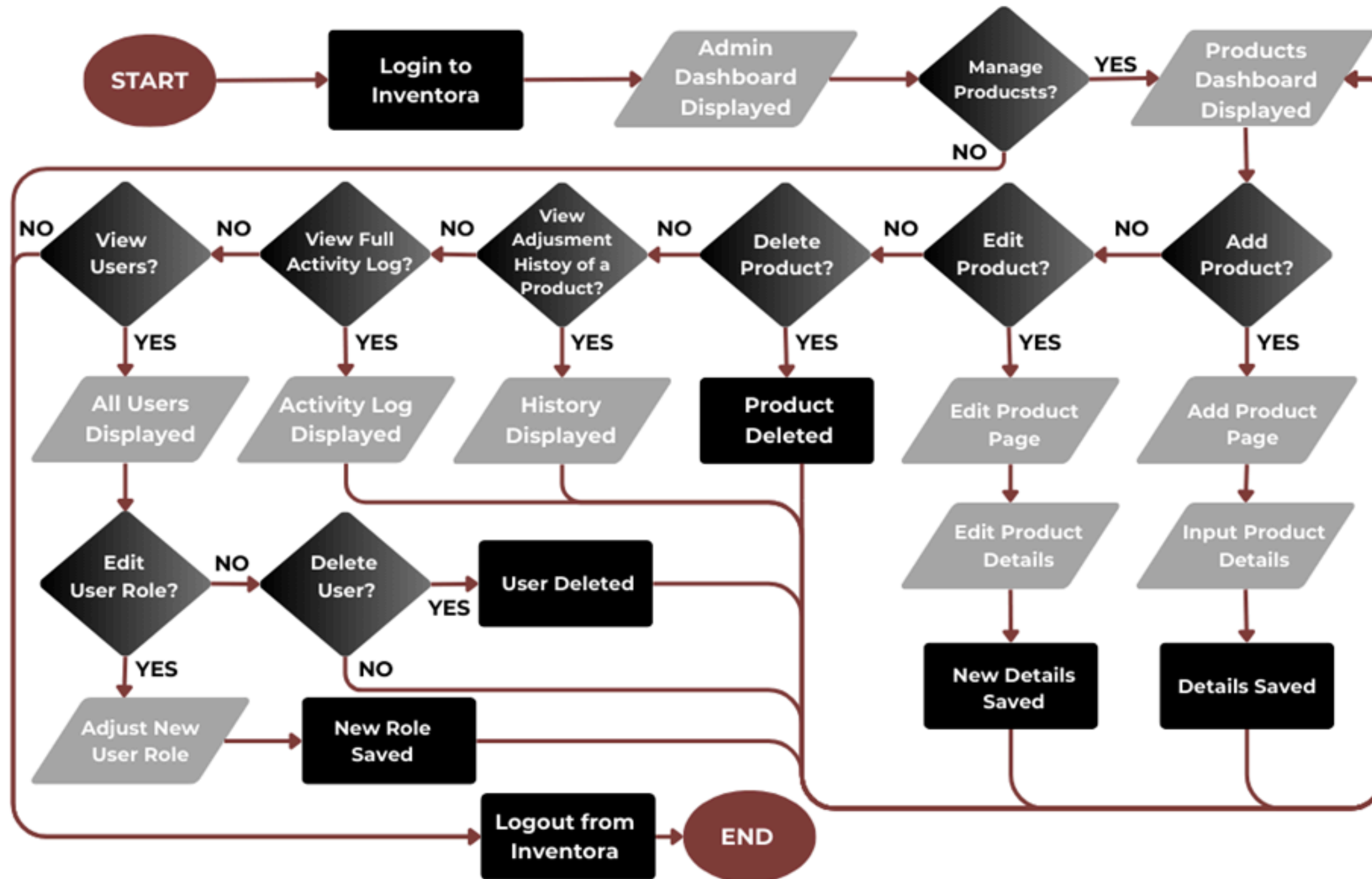
B. USERS AND ACCESS CONTROL

ROLE	ACCESS	CAPABILITIES	USE CASE
ADMIN	Full access to the system, including managing products, inventory adjustments, and user accounts.	• Perform CRUD (Create, Read, Update, Delete) operations on all data. • Manage system users, including creating new users and assigning roles. • View detailed activity logs and system reports.	Suitable for managers or owners overseeing the entire inventory system.
STAFF	Limited to viewing products and inventory adjustments.	• View product details. • Adjust stock quantities by adding or removing stock.	Suitable for employees handling day-to-day inventory operations.

II. PROCESSES

A. FLOWCHART

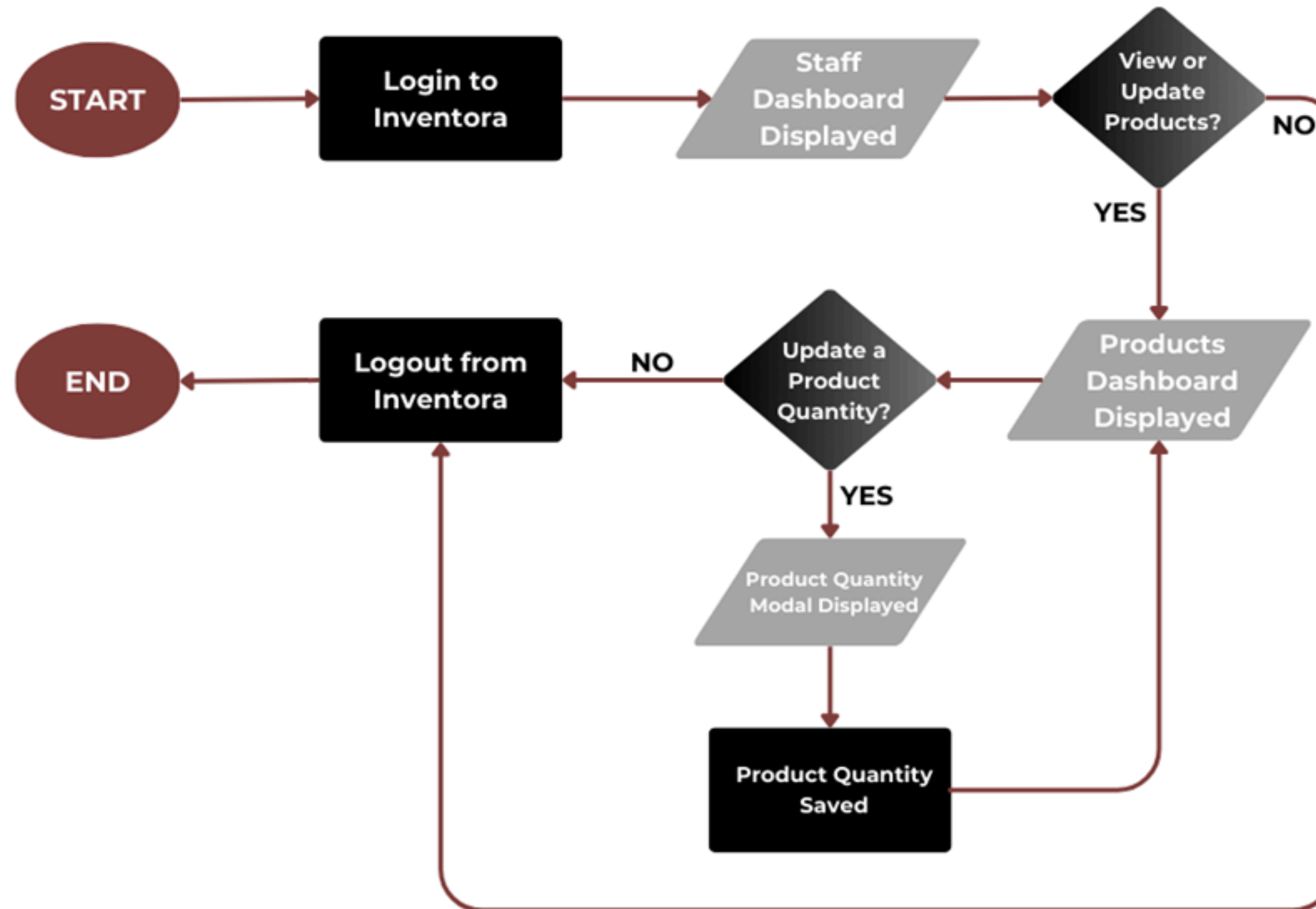
ADMIN Flowchart



II. PROCESSES

A. FLOWCHART

STAFF Flowchart



III. FEATURES IMPLEMENTATION

A. REPORTS/DATA IN DATABASE VIEWS

Our database includes several views and a materialized view to improve reporting and provide helpful insights. The `product_summary` view gives a detailed overview of products and their inventory adjustments. It includes data like product ID, name, category, current stock quantity, and the total quantities added or removed from stock. This is especially helpful for tracking product trends and managing stock flow.

The `activity_log_summary` view focuses on summarizing user activities, such as user ID, username, total actions performed, and the tables and columns that were modified. This view serves as a simple audit trail, making it easier for admins to monitor system activities.

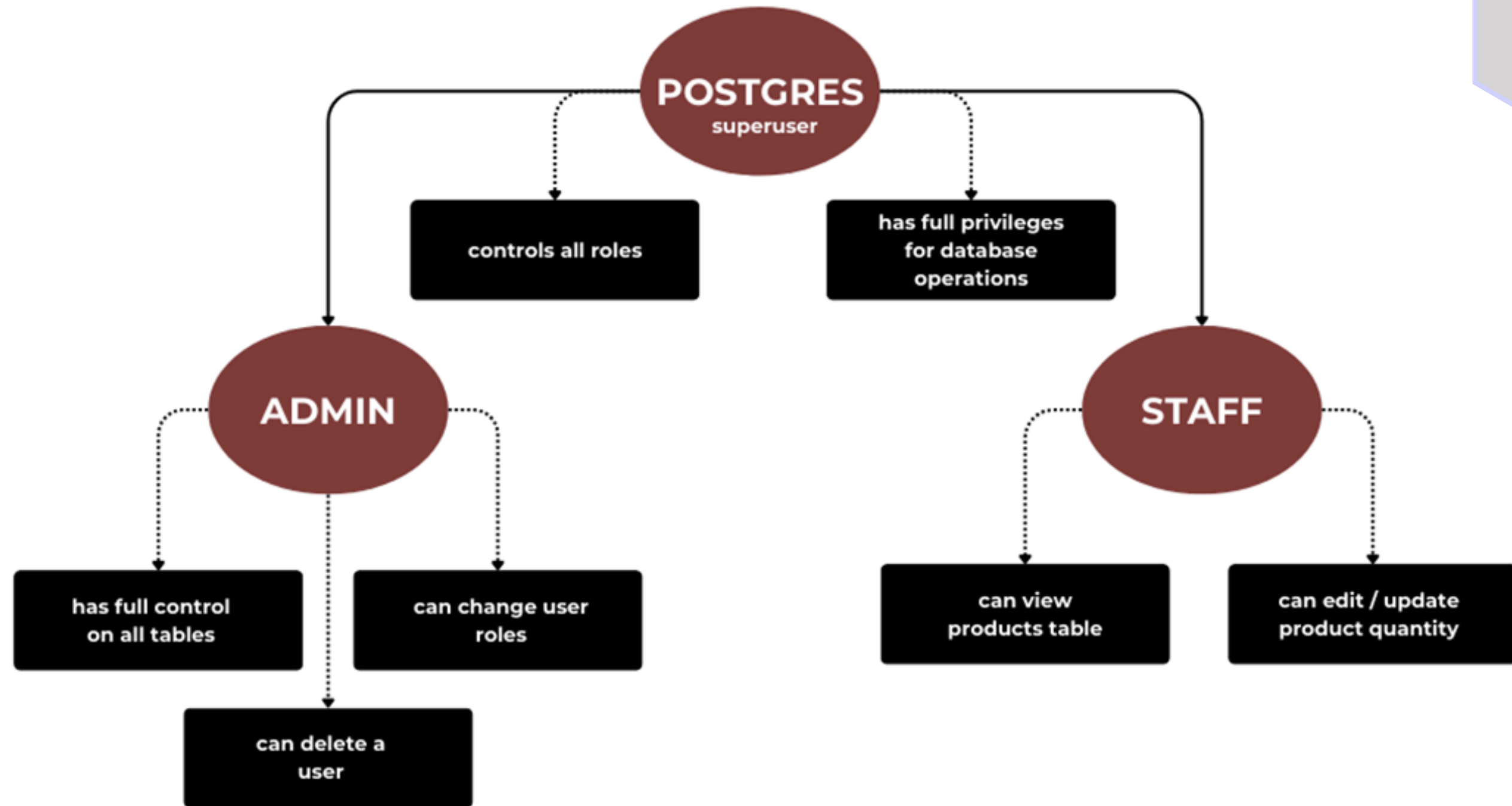
Lastly, the `mview_stock_critical` materialized view keeps track of products with low stock levels by displaying product ID, name, and current stock quantity for any item with less than 10 units. This view is super handy for identifying which products need restocking right away. Together, these views and the materialized view help keep the system organized and efficient.

B. FUNCTIONS AND THEIR TASKS

FUNCTION	PURPOSE	TASK	USE CASE
calculate_stock_value (product_id INT)	Computes the total value of stock for a specific product.	Multiplies stock quantity by price per unit and returns the result.	Supports financial analysis and product valuation.
get_adjustment_history (product_id INT)	Retrieves the inventory adjustment history for a specific product.	Returns details of all adjustments (added/removed quantities, type, date, and user responsible).	Tracks the historical changes in product stock.

C. ROLE-BASED ACCESS CONTROL (RBAC) DIAGRAM

RBAC Diagram



D. DATABASE INDEXING:

QUERY PERFORMANCE COMPARISON

QUERY	WITHOUT INDEX	WITH INDEX
CREATE INDEX idx_inventory_adjustments_product_id ON inventory_adjustments(product_id);	Planning Time: 0.062 ms Execution Time: 0.027 ms	Planning Time: 0.066 ms Execution Time: 0.022 ms
CREATE INDEX idx_products_stock_quantity ON products(stock_quantity);	Planning Time: 0.075 ms Execution Time: 0.029 ms	Planning Time: 0.085 ms Execution Time: 0.012 ms
CREATE INDEX idx_product_name_trgm ON products USING gin (product_name gin_trgm_ops);	Planning Time: 0.085 ms Execution Time: 0.041 ms	Planning Time: 0.081 ms Execution Time: 0.035 ms

E. ENTITY RELATIONSHIP DIAGRAM



F. ACTIVITY LOGGING IMPLEMENTATION

We implemented activity logging in our system using PostgreSQL triggers and functions to automatically track changes in key tables like products and inventory_adjustments. A central function, `log_activity()`, handles logging for INSERT, UPDATE, and DELETE actions. It records the table name, action type, user ID, timestamp, and details of modified columns. The user ID is fetched from a session variable (`myapp.user_id`), ensuring each action is linked to the user performing it.

For inventory changes, we created a dedicated function, `track_inventory_adjustments()`, which logs stock additions and removals in the inventory_adjustments table. This function captures details like the product ID, quantity added or removed, and the user responsible for the adjustment. All activity is stored in the activity_logs table for auditing purposes.



IV. TEAM COMPOSITIONS



Ushyne B. Esclamado



Jusalyn A. Gimao



Mark Daniel M. Jamisola

THANK YOU

